

COMP2522

Lesson 11: Unit Testing

LEARNING OUTCOMES

- Unit Testing
- Regression Testing
- TDD
- JUnit

TO SELL, YOU MUST PROVIDE TESTS

- Books / articles to read
- Examples of very small apps

UNIT TESTING

- Testing is a very important part of software development
- There are many types of testing:
<https://www.softwaretestinghelp.com/types-of-software-testing>
- Unit tests are software programs that test “units”; for example, that test the methods of our classes
- We write unit tests for several reasons:
 1. To ensure the method does what it should
 2. To ensure the method fails as it should (e.g. bad parameters → exceptions thrown)
 3. To check that our code still works even after it is modified; tests that *used to pass* either:
(a) still pass, or (b) now fail...but they show us what needs fixing! We are no longer “afraid” to make changes to our working code.

UNIT TESTING

- We will write tests for our constructors and methods
- Each test will eventually pass or fail
- JUnit is Java's unit-testing framework
- We will use assertions in our tests, which return true or false values
- Example: if we set a Person object's first name to "tiger" via a `setFirstName()` method, the `getFirstName()` method should return the string "tiger"
- Our assert statements can compare an expected value (e.g. "tiger") with the actual value (e.g. `person.getFirstName()`)
- If the actual value is different than the expected value, we would get a failed test
- If the actual value is the same as the expected value, we would get a passed test

JUNIT ASSERTIONS

- JUnit 5 assertions:

<https://junit.org/junit5/docs/current/api/org.junit.jupiter.api/org/junit/jupiter/api/Assertions.html>

JAVA CLASS AND ITS UNIT TEST

- Let's create a little class hierarchy (see next slides)
- Extend an abstract class
- Implement an interface
- Create some constructors and methods
- Create a concrete Dog class

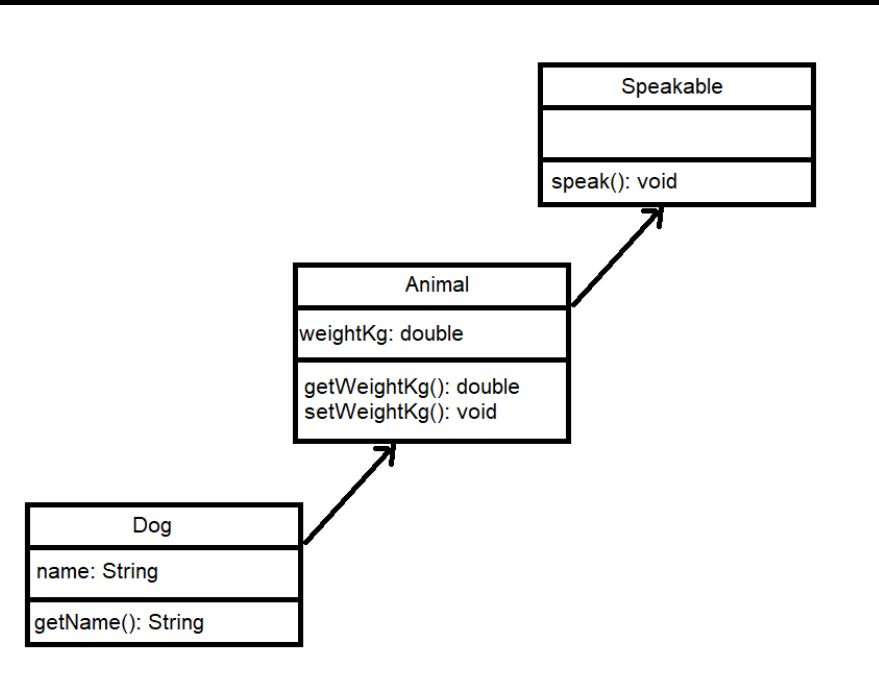
Test the Dog class

- I put the cursor beside the word “Dog” in the dog class definition and then typed **Alt-Enter** then chose **Create Test**
- I had to choose “Fix” because I had the error message “JUnit5 library not found in the module”


```
package ca.bcit.comp2601.animalkingdom;
```

```
public interface Speakable
```

```
{
    void speak(final String vocalization, final int numTimes);
}
```



```
package ca.bcit.comp2601.animalkingdom;
public abstract class Animal implements Speakable
{
    private double weightKg;

    public Animal(final double weightKg)
    {
        validateWeightKg(weightKg);
        this.weightKg = weightKg;
    }
    @Override
    public void speak(final String vocalization, final int numTimes)
    {
        for(int i = 0; i < numTimes; i++)
        {
            System.out.println(vocalization);
        }
    }
    private static void validateWeightKg(final double weightKg)
    {
        if(weightKg <= 0.0)
        {
            throw new IllegalArgumentException("invalid weight");
        }
    }
    public void setWeightKg(final double weightKg)
    {
        validateWeightKg(weightKg);
        this.weightKg = weightKg;
    }
    public double getWeightKg()
    {
        return weightKg;
    }
}
```

```
package ca.bcit.comp2601.animalkingdom;
```

```
8
```

```
public class Dog extends Animal
```

```
{
    private final String name;

    public Dog(final String name, final double
weightKg)
    {
        super(weightKg);

        if(name == null || name.isBlank())
        {
            throw new IllegalArgumentException("invalid
name");
        }

        this.name = name;
    }

    public String getName()
    {
        return name;
    }
}
```

```
package
ca.bcit.comp2601.animalkingdom;
```

```
public class Main
```

```
{
    public static void main(final String[] args)
    {
        Dog d = new Dog("rocky", 40.0);
        d.speak("woof", 4);
    }
}
```


FINALLY: WRITE SOME TESTS

- Imagine setting and getting the Dog object's name; if there were an error, it's not immediately clear whether the error was in the setting (e.g. mutator or constructor) or in the getting. Therefore let's unify these two methods into a single **name()** test inside the newly-created DogTest.java file:

```
@Test
void name()
{
    Dog dog;
    dog = new Dog("rocky", 40.0);
    assertEquals(dog.getName(), "rocky");    // passed test
    assertEquals(dog.getName(), "Rocky");    // failed test
}
```

MORE TESTS

- Imagine setting and getting the Dog object's name; if there were an error, it's not immediately clear whether the error was in the setting (e.g. mutator or constructor) or in the getting. Therefore let's unify these two methods into a single **name()** test inside the newly-created DogTest.java file:

```
@Test
void name()
{
    Dog dog;
    dog = new Dog("rocky", 40.0);
    assertEquals(dog.getName(), "rocky");           // passed test
    assertEquals(dog.getName(), "Rocky");           // failed test

    dog.setName("banjo");
    assertEquals(dog.getName(), "banjo");           // note: I added this later to the Dog class
                                                    // passed test
}
```

TESTING VOID PRINT METHOD

- The unit test for speak() is not trivial to test (testing what is printed on the console is not trivial)
- Rather than System.out.println'ing our parameter in the speak() method, **let's change it to return that string instead**
- Update (a) the Speakable interface speak() signature, (b) the Animal speak() implementation, and (c) the Main main() method call
- Now we can test the speak() method in DogTest's speak() test

DogTest.java

```
@Test
void testSpeak()
{
    final Dog dog;
    dog = new Dog("rocky", 40.0);
    assertEquals(dog.speak("woof", 4), "woof\nwoof\nwoof\nwoof\n");
    assertEquals(dog.speak("arf", 3), "arf\narf\narf\n");
}
```

Dog.java

```
@Override
public String speak(final String vocalization,
                    final int numTimes)
{
    final StringBuilder sb;
    sb = new StringBuilder();
    for(int i = 0; i < numTimes; i++)
    {
        sb.append(vocalization);
        sb.append("\n");
    }
    return sb.toString();
}
```

CODE LIKES TO HAVE A NAME

```
bar()    <-- test this
{
    do this
    do that
    do something else
}

foo()    <-- not this
{
    for(int i = 0; i < 1000000; i++)
    {
        // do this
        // do that
        // do something else
        bar()
    }
}
```

RUNNING TESTS

- Rather than running single tests, we can run all the tests in DogTest at once
- Now that all our tests are passing, we know that proper input to our methods and constructors, produces correct results
- Next let's rewrite our unit tests to show that improper input will produce the expected exceptions being
- Our new, updated test will assert that the exception thrown:
(a) is of the expected type (e.g. IllegalArgumentException)
and (b) has the proper message inside it (e.g. "Invalid weight kg"). If either (a) or (b) fails, our test will fail.

IMPROVED EXCEPTION HANDLING AND TESTS

- Since the chance is high that the user (i.e. not the developer) could enter a bad name for a Dog, let's make our own **checked** Exception type and ensure it's thrown whenever a Dog is trying to be given a name of null or blank.
- Update your code to throw this new checked exception type.
- try/catch where necessary now.
- Here is the updated test:

```
@Test
void badConstructor()
{
    final BadNameException e;

    e = assertThrows(BadNameException.class, ()->new Dog("", 40.0));
    assertEquals("invalid name", e.getMessage());
}
```

REFACTORED CONSTRUCTOR TESTS

```
@Test
void constructor_nullName_throwsIllegalArgumentException() // give EXTREMELY-specific test names
{
    final IllegalArgumentException e;

    e = assertThrows(IllegalArgumentException.class, ()->new Dog(null, 40.0));
    assertEquals("name must not be null", e.getMessage());
}
```

```
@Test
void constructor_blankName_throwsBadNameException()
{
    final BadNameException e;

    e = assertThrows(BadNameException.class, ()->new Dog("", 40.0));
    assertEquals("name must not be blank", e.getMessage());
}
```


REFACTORED CONSTRUCTOR TESTS

Even better, remove the `@Test` annotation from the bottom method (so it won't run itself as a test) and call that method with a parameter, giving us a reusable test with minimal code:

```
@Test
void constructor_variousBlankNames_throwBadNameException ()
{
    checkBlankName("");
    checkBlankName("\n");
    checkBlankName("\t");
    checkBlankName("\t \n \t"); // etc....
}

void checkBlankName(final String attemptedName)
{
    final BadNameException e;

    e = assertThrows(BadNameException.class, ()->new Dog(attemptedName, 40.0));
    assertEquals("name must not be blank", e.getMessage());
}
```

REFACTORED CONSTRUCTOR TESTS

- Next we need to ensure that Dog's setName() method also throws the same Exceptions in the same cases, similarly to how we centralized the weight validation in the Animal class (see earlier slides).

TEST-DRIVEN DEVELOPMENT

- Many of you have already seen TDD in action
- TDD involves writing the a test first, and then immediately writing the code to pass that test
- The advantage of having the tests in place *first* is that errors can be found earlier in the software-development process

MORE UNIT-TESTING BEST PRACTICES

- Keep the tests and code in separate folders
- Keep the tests and code in the same package
- Avoid `assertTrue()` and `assertFalse()`...they are too unclear
- Use code coverage (File / Run / Run with Coverage)

FINAL EXAMPLE

```
class BankAccountTest
{
    @Test
    void testGoodTransactions() throws IllegalTransactionException
    {
        BankAccount b;

        b = new BankAccount("abc123", 500.00);
        assertEquals(b.getBalanceCad(), 500.00, 0.01); // within 0.01 Cad
    }

    @Test
    void testBadTransactions() throws IllegalTransactionException
    {
        IllegalTransactionException ex;
        BankAccount b;

        b = new BankAccount("abc123", 500.00);

        ex = assertThrows(IllegalTransactionException.class, ()->b.setAccountNumber(""));
        assertEquals(ex.getMessage(), "no no");
    }
}
```

POSITIVE AND NEGATIVE TESTING

21

```
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class BookTest{
    Book b1;
    Book b2;
    @BeforeEach
    void setUp() {
        try {
            b1 = new Book("harry potter", 1995);
            b2 = new
Book("01234567890123456789012345678901234", 1954); //
MAX_TITLE_LEN
        } catch (final Exception ex) {
            fail();
        }
    }
    @AfterEach
    void tearDown() {
        b1 = null;
    }
}
```

```
@Test
void testTitle() {
    assertEquals("harry potter", b1.getTitle());
    assertEquals("01234567890123456789012345678901234",
b2.getTitle());
    badTitle(null);
    badTitle("");
    badTitle(" ");
    badTitle("bcit"); // bcit in any letter casing isn't allowed in the title
    badTitle("isn't BCIT fun?");
    badTitle("0123456789012345678901234567890123456789012345"); //
Book::MAX_TITLE_LEN
}
@Test
void testYearPublished() {
    assertEquals(1995, b1.getYearPublished());
    assertEquals(1954, b2.getYearPublished());
}
void badTitle(final String title) {
    final IllegalArgumentException ex;
    ex = assertThrows(IllegalArgumentException.class, ()->new Book(title,
2000));
    assertEquals("Invalid title: " + title, ex.getMessage());
}
}
```

DESIGN BY CONTRACT

- design by contract
- `int getRangeNMiOfEntity(final int dgTN)`
- `{`
- `assert(dgTN >= 0);`
- `assert(dgTN < 360);`
-
- `....`
- `assert(ret > 0);`
- `assert(ret < MAX_RADAR_RNG_NMI);`
- `return ret;`
- `}`